Onyx — ImageKit reference

Onyx · multi-process OS on Raspberry Pi 4

*The reference of **ImageKit** (`SD:/lib/imagekit.so`, `user/Kits/imagekit`): what it is for, how a program uses it, and every operation it exposes. The operations' part is made from the kit's headers by `tools/docgen/kitdocs.py` — the headers are the source. Overview of all the kits: The Kits.*

Contents

1. What it is
2. Using it
3. Index
4. `imagekit/imagekit.h`

What it is

ImageKit is the system's one picture codec: BMP, GIF, PNG, JPEG, PCX and WebP read; PNG, JPEG, BMP and GIF written; resizing, turning, the photo adjustments.

Include	`#include "imagekit/imagekit.h"`
Link	`lib/imagekit.imp.a`
Library	`SD:/lib/imagekit.so` — 46 entries in its table (`user/Kits/imagekit/imagekit.abi`, append-only)
Sources	`user/Kits/imagekit/`

Using it

ImageKit is the system's one picture codec: BMP, GIF, PNG, JPEG, PCX and WebP read; PNG, JPEG, BMP and GIF written.

A picture read, turned, saved in another format:

```
#include "imagekit/imagekit.h"

ik_image *im = ik_load ("SD:/photos/cat.jpg", 0);
if (im)
{
    int w = ik_width (im), h = ik_height (im);
    unsigned *px = ik_pixels (im);                    // w * h pixels, 0xAARRGGBB
    ik_rotate (im, 1);                                // a quarter turn clockwise
    ik_save (im, "SD:/photos/cat.png", 90);           // the format: the extension's
    ik_image_free (im);
}
```

Pixels of the program's own, saved:

```
ik_image *shot = ik_image_from (pixels, width, height, width);
ik_save (shot, "SD:/screenshots/shot.png", 0);
ik_image_free (shot);
```

The formats, asked — a file chooser's filter follows what the library knows:

```
struct ik_format f[16];
int n = ik_formats (f, 16);
```

Resizing (`ik_scale`), flipping (`ik_flip`), the photo adjustments and the EXIF orientation are in the same header.

Index

Everything the headers declare, in their order — the details are in each header's part below.

Name	What it does	Header
`ik_free`	a buffer the library returned	`imagekit.h`
`ik_image_new`	transparent black; 0: no memory, a bad size	`imagekit.h`
`ik_image_from`	a copy of pixels (stride: in pixels)	`imagekit.h`
`ik_image_copy`	a new picture with the same pixels; 0: no picture, no memory	`imagekit.h`
`ik_image_free`	the picture and its pixels freed (0: nothing)	`imagekit.h`
`ik_width`	in pixels (0: no picture)	`imagekit.h`
`ik_height`	in pixels (0: no picture)	`imagekit.h`
`ik_format`	IK_ARGB8	`imagekit.h`
`ik_pixels`	its w * h pixels (until the image changes size)	`imagekit.h`
`ik_fill`	every pixel set to that 0xAARRGGBB value	`imagekit.h`
`ik_opaque`	every pixel's alpha 255	`imagekit.h`
`ik_flatten`	laid on a colour, the result opaque	`imagekit.h`
`ik_has_alpha`	1: a pixel is not opaque	`imagekit.h`
`ik_formats`	how many there are (out: up to max)	`imagekit.h`
`ik_info`	(a type)	`imagekit.h`
`ik_is_image_name`	by its extension (bmp gif png jpg jpeg jpe pcx webp)	`imagekit.h`
`ik_probe`	1 / 0: not a picture this reads	`imagekit.h`
`ik_load`	the picture (an animation: its first	`imagekit.h`

Name	What it does	Header
	frame); 0	
`ik_load_mem`	the same from a file's n bytes in memory (flags: ignored, no orientation applied); 0	`imagekit.h`
`ik_load_preview`	the camera's preview, turned as seen; 0: none	`imagekit.h`
`ik_load_format`	the last picture read: "PNG", "JPEG"... (": none)	`imagekit.h`
`ik_frames_load`	the file read, every frame decoded (a still picture: one frame); 0: not read	`imagekit.h`
`ik_frames_load_mem`	the same from a file's n bytes in memory; 0	`imagekit.h`
`ik_frames_count`	how many frames it holds (0: none)	`imagekit.h`
`ik_frames_pixels`	frame i: w * h pixels (the library's)	`imagekit.h`
`ik_frames_delay`	how long it shows, ms (0: a still picture)	`imagekit.h`
`ik_frames_width`	every frame's width, pixels	`imagekit.h`
`ik_frames_height`	every frame's height, pixels	`imagekit.h`
`ik_frames_free`	the frames it still holds and the handle freed (0: nothing)	`imagekit.h`
`ik_frames_take`	The frames handed over to the caller (px[i]	`imagekit.h`
`ik_inflate`	A deflate stream inflated (zlib	`imagekit.h`
`ik_encode`	0 / -1	`imagekit.h`
`ik_save`	the format from the path's extension → 0 / -1	`imagekit.h`
`ik_encode_pixels`	Raw pixels encoded (stride	`imagekit.h`
`ik_scale`	The low-level one	`imagekit.h`
`ik_scale_rgb`	The same for pixels whose top byte is not an alpha (0x00RRGGBB, a window's canvas, a photo)	`imagekit.h`

Name	What it does	Header
`ik_resize`	a new picture of that size	`imagekit.h`
`ik_fit`	within the box, proportions kept (grow 0: never larger)	`imagekit.h`
`ik_cover`	the middle cut to that shape and brought to it (a thumbnail)	`imagekit.h`
`ik_crop`	in place → 0 / -1	`imagekit.h`
`ik_rotate`	clockwise, in place	`imagekit.h`
`ik_flip`	0: left-right, 1: top-bottom	`imagekit.h`
`ik_orient`	1..8 applied	`imagekit.h`
`ik_straighten`	a small angle, enlarged so no corner is empty (the result opaque)	`imagekit.h`
`ik_adjust`	(a type)	`imagekit.h`
`ik_adjust_apply`	in place (the alpha kept)	`imagekit.h`
`ik_adjust_auto`	"enhance": from the picture's histogram	`imagekit.h`
`ik_filter_name`	"Black and white" ...	`imagekit.h`

`imagekit/imagekit.h`

imagekit.h -- ImageKit: pictures, one copy for the whole system (SD:/lib/imagekit.so; the shared libraries: docs/03 section 5.6). A program links lib/imagekit.imp.a and calls plain C functions (ik_*). The first version (2026-10-05) is the base every program that shows or writes a picture needs; the layers, the masks and the brushes of the photo editor come on top of it later.

```
  images      a picture in memory (ik_image): w x h pixels 0xAARRGGBB, straight
alpha, rows one
              after the other
  reading     BMP, GIF (animated too), PNG, JPEG, PCX, WebP -- from a file or from
memory; what a
              file says without decoding it (its size, the camera's orientation,
date, model);
              the camera's own preview
  writing     PNG (zlib's compression, through FileKit), JPEG, BMP, GIF -- to a file
or to memory
  transforms  ONE resize, right in alpha (the average of what a pixel covers when
smaller,
              bilinear when larger); fit, cover, crop, quarter turns, flips, the
camera's
              orientation applied, straightened by a small angle
  adjustments exposure, contrast, highlights, shadows, saturation, warmth, sharpness
and six
              filters as one tone curve; "enhance" from the histogram (Photos' own)
```

Every call takes integers and pointers only: a program built without the FPU calls it. An ik_image is opaque (its format may grow: 16 bits a channel later); a buffer the library returns (void **out) is freed with ik_free.

MIT License -- Copyright (c) 2026 Stéphane Wegener and the Onyx contributors. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.

```
void ik_free (void *p);             // a buffer the library returned
```

a picture in memory

```
#define IK_ARGB8    1           // 0xAARRGGBB, 8 bits a channel, straight alpha (the only format today)

typedef struct ik_image ik_image;
ik_image *ik_image_new (int w, int h);              // transparent black; 0: no memory, a bad size
ik_image *ik_image_from (const unsigned *px, int w, int h, int stride); // a copy of pixels (stride: in pixels)
ik_image *ik_image_copy (const ik_image *im);   // a new picture with the same pixels; 0: no picture, no memory
void ik_image_free (ik_image *im);  // the picture and its pixels freed (0: nothing)
int ik_width (const ik_image *im);  // in pixels (0: no picture)
int ik_height (const ik_image *im); // in pixels (0: no picture)
int ik_format (const ik_image *im);             // IK_ARGB8
unsigned *ik_pixels (ik_image *im);             // its w * h pixels (until the image changes size)
void ik_fill (ik_image *im, unsigned argb); // every pixel set to that 0xAARRGGBB value
void ik_opaque (ik_image *im);                  // every pixel's alpha 255
void ik_flatten (ik_image *im, unsigned rgb);           // laid on a colour, the result opaque
int ik_has_alpha (const ik_image *im);              // 1: a pixel is not opaque
```

the formats, asked (version 2)

A program lists what the library reads and writes instead of keeping its own list: a format the library learns is one every program has.

```
struct ik_format
{
    char name[12];              // "PNG", "JPEG", "GIF", "BMP", "PCX", "WebP"
    char extensions[40];            // "jpg jpeg jpe" (lower case, a space between)
    int  can_read, can_write;
    int  alpha;             // it keeps transparency
    int  animated;              // it may hold several frames (read as ik_frames)
    int  reserved[4];
};
int ik_formats (struct ik_format *out, int max);        // how many there are (out: up to max)
```

reading

```
#define IK_ORIENT   1           // the camera's orientation (EXIF) applied: as the photo is seen

struct ik_info
{
    int   w, h;             // as stored (before the orientation); 0: not known without decoding
    int   orientation;          // 1..8 (EXIF), 1: as stored
    long long taken;            // when (seconds, local time, from 1970), 0: not known
    char      camera[64];           // "Google Pixel 8" ("": not known)
    char      exposure[64];         // "f/1.7 · 1/640 s · ISO 50 · 6.9 mm"
    int   has_preview;          // the camera's own small picture is in the file (ik_load_preview)
    int   reserved[8];
};
int ik_is_image_name (const char *name);            // by its extension (bmp gif png jpg jpeg jpe pcx webp)
int ik_probe (const char *path, struct ik_info *out);       // 1 / 0: not a picture this reads
ik_image *ik_load (const char *path, int flags);        // the picture (an animation: its first frame); 0
ik_image *ik_load_mem (const void *data, unsigned n, int flags);    // the same from a file's n bytes in memory (flags: ignored, no orientation applied); 0
ik_image *ik_load_preview (const char *path);           // the camera's preview, turned as seen; 0: none
const char *ik_load_format (void);              // the last picture read: "PNG", "JPEG"... (": none)
```

An animation (a GIF): every frame.

```
typedef struct ik_frames ik_frames;
ik_frames *ik_frames_load (const char *path);   // the file read, every frame decoded (a still picture: one frame); 0: not read
ik_frames *ik_frames_load_mem (const void *data, unsigned n);   // the same from a file's n bytes in memory; 0
int ik_frames_count (const ik_frames *f);   // how many frames it holds (0: none)
const unsigned *ik_frames_pixels (const ik_frames *f, int i);   // frame i: w * h pixels (the library's)
int ik_frames_delay (const ik_frames *f, int i);        // how long it shows, ms (0: a still picture)
int ik_frames_width (const ik_frames *f);   // every frame's width, pixels
int ik_frames_height (const ik_frames *f);  // every frame's height, pixels
void ik_frames_free (ik_frames *f); // the frames it still holds and the handle freed (0: nothing)
```

The frames handed over to the caller (px[i]: new unsigned[w * h], the caller's to delete []; delay[i]: ms): up to max of them → how many; f keeps none of those. (UIKit's img_load is this.)

```
int ik_frames_take (ik_frames *f, unsigned **px, int *delay, int max);
```

A deflate stream inflated (zlib: with its header, as PNG's; else raw): a new unsigned char[] of *out_n bytes (the caller's to delete []), or 0. (UIKit's img_inflate; a new program takes FileKit's.)

```
unsigned char *ik_inflate (const void *data, unsigned n, int zlib, unsigned *out_n);
```

writing

format: "png", "jpg" / "jpeg", "bmp", "gif" (letters' case ignored). quality: JPEG's 1..100 (0: 90). A picture with transparency keeps it in PNG; GIF keeps its clear pixels (alpha < 128) clear and the others opaque; JPEG and BMP lay it on white.

```
int ik_encode (const ik_image *im, const char *format, int quality, void **out,
unsigned *out_n);   // 0 / -1
int ik_save (const ik_image *im, const char *path, int quality);    // the format
from the path's extension -> 0 / -1
```

Raw pixels encoded (stride: pixels a row). alpha: 1 kept where the format has it (PNG: RGBA), 0 not (PNG: RGB, whatever the top bytes hold), -1 kept when a pixel is not opaque. (img/pngsave.hpp's encoders are this, in the programs built with PNGSAVE_USE_IMAGEKIT.)

```
int ik_encode_pixels (const unsigned *px, int w, int h, int stride, const char
*format, int quality, int alpha,
              void **out, unsigned *out_n);
```

transforms

The low-level one: the rectangle (sx, sy, sw, sh) of src (src_w x src_h, src_stride pixels a row) brought to dst (dw x dh, dst_stride pixels a row). Right in alpha (the colours weighed by it). A big shrink reads at most ~8 × 8 of the pixels each result covers (a 24 Mpx photo's thumbnail stays quick).

```
void ik_scale (const unsigned *src, int src_w, int src_h, int src_stride, int sx, int
sy, int sw, int sh,
           unsigned *dst, int dst_stride, int dw, int dh);
```

The same for pixels whose top byte is not an alpha (0x00RRGGBB, a window's canvas, a photo): the colours alone; the result's top byte is 0.

```
void ik_scale_rgb (const unsigned *src, int src_w, int src_h, int src_stride, int sx,
int sy, int sw, int sh,
           unsigned *dst, int dst_stride, int dw, int dh);
ik_image *ik_resize (const ik_image *im, int w, int h);     // a new picture of that
size
ik_image *ik_fit (const ik_image *im, int max_w, int max_h, int grow);  // within the
box, proportions kept (grow 0: never larger)
ik_image *ik_cover (const ik_image *im, int w, int h);      // the middle cut to that
shape and brought to it (a thumbnail)
int ik_crop (ik_image *im, int x, int y, int w, int h);     // in place -> 0 / -1
int ik_rotate (ik_image *im, int quarter_turns);        // clockwise, in place
int ik_flip (ik_image *im, int vertical);           // 0: left-right, 1: top-bottom
int ik_orient (ik_image *im, int exif_orientation);     // 1..8 applied
int ik_straighten (ik_image *im, int millidegrees);     // a small angle, enlarged so
no corner is empty (the result opaque)
```

adjustments (Photos')

```
#define IK_FILTER_NONE      0
#define IK_FILTER_BW        1
#define IK_FILTER_WARM      2
#define IK_FILTER_COOL      3
#define IK_FILTER_VINTAGE   4
#define IK_FILTER_VIVID     5
#define IK_FILTERS      6

struct ik_adjust
```

```
{
    int exposure, contrast, highlights, shadows, saturation, warmth;    // -100..100,
0: as it is
    int sharpness;                                  // 0..100
    int filter;                                 // IK_FILTER_*
    int reserved[8];                                // 0
};
void ik_adjust_apply (ik_image *im, const struct ik_adjust *a); // in place (the
alpha kept)
void ik_adjust_auto (const ik_image *im, struct ik_adjust *out);    // "enhance":
from the picture's histogram
const char *ik_filter_name (int filter);            // "Black and white" ...

}
#endif
```